The objective of Project 2 was to adapt the solution developed in Project 1 to a fully portable, vendor-agnostic architecture capable of running in any cloud platform that supports Docker containers and Kubernetes. This modernization removes the dependency on proprietary Azure services and enables deployment in heterogeneous environments.

The redesigned solution consists of several containerized services orchestrated by Kubernetes: the application server, a local caching server, a media storage subsystem backed by persistent volume and a local database. These components together implement the Lego Sets management platform.

1.1. Main Components

Application Server

The backend, originally deployed in Azure App Service, was containerized using a custom Dockerfile and is now executed as a Kubernetes Deployment with multiple replicas. The server is stateless; therefore, horizontal scaling can be achieved simply by increasing the number of replicas. All configurations (database endpoints, Redis host, media storage path, etc.) are passed through environment variables so the application can run in any Kubernetes-compatible environment.

Local Redis Cache

Caching relayed on Azure Cache for Redis was replaced by an internal Redis server, packaged as a Docker container and deployed inside the Kubernetes cluster. The backend communicates with Redis through a Cluster IP Service, ensuring stable internal DNS resolution. The caching layer preserves all optimization introduced in Project 1, session management using cookie-based authentication. This provides the same performance advantages without relying on any cloud-specific service.

Persistent Volume for Media Storage

Media files were previously stored in Azure Blob Storage. To remove platform dependencies, a Persistent Volume Claim (PVC) was created, and the application server mounts this volume directly into the container’s filesystem.

All uploaded media is therefore stored and served locally from the persistent volume, which guarantees:

- Durability across pod restarts

- Higher portability
- Filesystem-level simplicity
- Complete independence from object-storage services

· **Local Database**

To increase portability, the system was extended to support MongoDB running as a container inside the Kubernetes cluster.
A dedicated Deployment and PVC ensure data durability, and a ClusterIP Service exposes MongoDB to the backend application.

2. Results

The load test for Project 2 was executed using the same methodology adopted in Project 1, allowing a consistent comparison between both architectures. All pods remained stable, and no timeouts, restarts or throttling events were observed.

Comparing these results with Project 1 highlights several improvements:

1. **No throttling or timeouts**, unlike Project 1 where Azure App Service and Cosmos DB occasionally introduced delays or RU-based limitations during intensive write operations.

2. **Lower and more stable latency**, as all components (backend, Redis, MongoDB, media storage) now run inside the same Kubernetes cluster, eliminating cross-service network overhead.

3. **More consistent media uploads**, with the PVC-based storage showing faster and more predictable performance compared to Azure Blob Storage.

4. **Fewer unintended errors**, suggesting the Kubernetes environment is less susceptible to transient behaviors caused by external dependencies.

Overall, the results show that the architecture developed in Project 2 is not only fully portable and cloud-agnostic, but also **more stable, faster and more predictable** than the original Azure-based solution. The consolidation of all services within Kubernetes significantly enhanced system responsiveness and eliminated the performance constraints observed in Project 1.

Files

Dockerfile - Files

docker-compose.yaml

```
version: '3.8'

services:
  mongo:
    image: mongo:7
    container_name: lego-mongo
    platform: linux/amd64
    restart: always
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: senhaSegura123
      MONGO_INITDB_DATABASE: ccdb
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--username", "admin", "--password", "senhaSegura123",
"--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - lego-network

  redis:
    image: redis:7-alpine
    container_name: lego-redis
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - lego-network
```

```
  webapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: lego-webapp
    environment:
      DB_URL: mongodb://admin:senhaSegura123@mongo:27017/ccdb?authSource=admin
      DB_NAME: ccdb
      DB_KEY: ""
      REDIS_HOST: redis
      REDIS_PORT: 6379
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-prod}
    ports:
      - "8080:8080"
    depends_on:
      mongo:
        condition: service_healthy
      redis:
        condition: service_healthy
    volumes:
      - media-storage:/app/media
    networks:
      - lego-network
    restart: unless-stopped

volumes:
  mongo-data:
    driver: local
  redis-data:
    driver: local
  media-storage:
    driver: local

networks:
  lego-network:
    driver: bridge
```

Dockerfile

```
FROM tomcat:10.1-jdk21-temurin

RUN rm -rf /usr/local/tomcat/webapps/*

COPY webapp/target/*.war /usr/local/tomcat/webapps/ROOT.war

EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Kubernetes

app-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: legoapp
spec:
 replicas: 1
 selector:
   matchLabels:
     app: legoapp
 template:
   metadata:
     labels:
       app: legoapp
   spec:
     containers:
       - name: legoapp
         image: cc2526acr.azurecr.io/lego-webapp:latest
         imagePullPolicy: Always
         ports:
           - containerPort: 8080
         env:
           # --- CONFIGURAÇÃO DO VOLUME ---
           - name: MEDIA_PATH
             value: /media

           # --- REDIS LOCAL (K8s Service) ---
           - name: REDIS_URL
             value: "redis://redis:6379"
           - name: REDIS_KEY
             value: ""

           # --- VARIAVEIS SENSIVEIS (Vêm do Secret 'lego-app-secrets') ---
           - name: DB_URL
             valueFrom:
               secretKeyRef:
                 name: lego-app-secrets
```

```
              key: DB_URL
          - name: DB_KEY
            valueFrom:
              secretKeyRef:
                name: lego-app-secrets
                key: DB_KEY
          - name: DB_NAME
            valueFrom:
              secretKeyRef:
                name: lego-app-secrets
                key: DB_NAME
          - name: BlobStoreConnection
            valueFrom:
              secretKeyRef:
                name: lego-app-secrets
                key: BlobStoreConnection
          - name: AZURE_STORAGE_CONTAINER
            valueFrom:
              secretKeyRef:
                name: lego-app-secrets
                key: AZURE_STORAGE_CONTAINER

        # --- VOLUMES E MONTAGENS ---
          - name: DB_URL
            value:
mongodb://admin:senhaSegura123@mongodb-service:27017/ccdb?authSource=admin
          - name: DB_NAME
            value: ccdb
          # se quiser desligar Cosmos de vez:
          - name: DB_KEY
            value: ""    #não necessário Mongo
        volumeMounts:
          - name: media-volume
            mountPath: /media
        resources:
          requests:
            memory: "256Mi"
            cpu: "100m"
          limits:
            memory: "512Mi"
            cpu: "200m"
        livenessProbe:
          httpGet:
            path: /rest/ctrl/version
            port: 8080
```

```
          initialDelaySeconds: 60
          periodSeconds: 10

    volumes:
      - name: media-volume
        persistentVolumeClaim:
          claimName: media-pvc
```

app-service.yaml

```
apiVersion: v1
kind: Service
metadata:
 name: legoapp-service
spec:
 type: LoadBalancer
 ports:
   - port: 80
     targetPort: 8080
 selector:
   app: legoapp
```

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: lego-webapp
 namespace: default
spec:
 replicas: 2
 selector:
   matchLabels:
     app: lego-webapp
 template:
   metadata:
     labels:
       app: lego-webapp
   spec:
```

```
    containers:
    - name: webapp
      image: cc2526acr.azurecr.io/lego-webapp:v1
      ports:
      - containerPort: 8080
      env:
      # --- REDIS LOCAL (K8s Service) ---
      # Este não é sensível, aponta para o serviço interno
      - name: REDIS_URL
        value: "redis://redis:6379"
      - name: REDIS_KEY
        value: ""

      # --- VARIAVEIS SENSIVEIS (Vêm do Secret 'lego-app-secrets') ---
      - name: DB_URL
        valueFrom:
          secretKeyRef:
            name: lego-app-secrets
            key: DB_URL
      - name: DB_KEY
        valueFrom:
          secretKeyRef:
            name: lego-app-secrets
            key: DB_KEY
      - name: DB_NAME
        valueFrom:
          secretKeyRef:
            name: lego-app-secrets
            key: DB_NAME
      - name: BlobStoreConnection
        valueFrom:
          secretKeyRef:
            name: lego-app-secrets
            key: BlobStoreConnection
      - name: AZURE_STORAGE_CONTAINER
        valueFrom:
          secretKeyRef:
            name: lego-app-secrets
            key: AZURE_STORAGE_CONTAINER

      resources:
        requests:
          memory: "256Mi"
          cpu: "100m"
        limits:
```

```
          memory: "512Mi"
          cpu: "200m"
      livenessProbe:
        httpGet:
          path: /rest/ctrl/version
          port: 8080
        initialDelaySeconds: 60
        periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
 name: lego-webapp-service
 namespace: default
spec:
 type: LoadBalancer
 selector:
   app: lego-webapp
 ports:
 - port: 80
   targetPort: 8080
   protocol: TCP
```

mongodb.yaml

```
apiVersion: v1
kind: Secret
metadata:
 name: mongodb-secret
type: Opaque
stringData:
 mongo-root-username: admin
 mongo-root-password: senhaSegura123
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: mongodb-pvc
spec:
 accessModes:
   - ReadWriteOnce
 resources:
   requests:
```

```
    storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
 name: mongodb
spec:
 replicas: 1
 selector:
   matchLabels:
     app: mongodb
 template:
   metadata:
     labels:
       app: mongodb   #label tem de bater certo com o selector do Service
   spec:
     containers:
       - name: mongodb
         image: mongo:7
         ports:
           - containerPort: 27017
         env:
           - name: MONGO_INITDB_ROOT_USERNAME
             valueFrom:
               secretKeyRef:
                 name: mongodb-secret
                 key: mongo-root-username
           - name: MONGO_INITDB_ROOT_PASSWORD
             valueFrom:
               secretKeyRef:
                 name: mongodb-secret
                 key: mongo-root-password
           - name: MONGO_INITDB_DATABASE
             value: ccdb
         volumeMounts:
           - name: mongodb-data
             mountPath: /data/db
     volumes:
       - name: mongodb-data
         persistentVolumeClaim:
           claimName: mongodb-pvc
---
apiVersion: v1
kind: Service
metadata:
```

```
 name: mongodb-service
spec:
 type: ClusterIP
 ports:
   - port: 27017
     targetPort: 27017
 selector:
   app: mongodb
```

pvc-media.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: media-pvc
spec:
 accessModes:
   - ReadWriteOnce
 resources:
   requests:
     storage: 5Gi
 storageClassName: managed-csi
```

redis-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: redis
 labels:
   app: redis
spec:
 replicas: 1
 selector:
   matchLabels:
     app: redis
 template:
   metadata:
     labels:
       app: redis
   spec:
     containers:
```

```
    - name: redis
      image: redis:7-alpine
      ports:
      - containerPort: 6379
      resources:
        requests:
          cpu: 100m
          memory: 100Mi
        limits:
          cpu: 250m
          memory: 256Mi
```

redis-service.yaml

```
apiVersion: v1
kind: Service
metadata:
 name: redis
spec:
 ports:
 - port: 6379
   targetPort: 6379
 selector:
   app: redis
```